

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра математического обеспечения и применения ЭВМ**

**ОТЧЕТ**  
**по практической работе №5**  
**по дисциплине «Системы параллельной обработки данных»**  
**Тема: Умножение матриц**

Студент гр. 1306

\_\_\_\_\_

Шаврин А.П.

Преподаватель

\_\_\_\_\_

Татаринов Ю.С.

Санкт-Петербург

2025

## **Цель работы.**

Разработать и провести сравнительный анализ последовательного и параллельного алгоритмов умножения квадратных матриц с использованием технологии MPI (Message Passing Interface) в среде C++.

## **Постановка задачи.**

**Вариант 2.** Выполнить задачу умножения двух квадратных матриц A и B размера  $m \times m$ , результат записать в матрицу C. Реализовать последовательный и параллельный ленточный алгоритм 2 (горизонтальные и вертикальные полосы) и провести анализ полученных результатов.

## **Выполнение работы.**

### **1. Реализация последовательного алгоритма умножения матриц**

Разработка программы началась с определения структуры данных — квадратной матрицы, представленной в виде одномерного массива `std::vector<int>` фиксированного размера  $N \times N$ . Для удобства доступа к элементам матрицы реализованы две вспомогательные функции `at`, позволяющие обращаться к элементу по индексам  $(i, j)$  и устанавливать его значения.

Для генерации исходных данных была создана функция: `generateRandomMatrix` которая формирует матрицу с псевдослучайными значениями, что обеспечивает воспроизводимость экспериментов и единообразие тестов.

Для реализации последовательного умножения матриц предусмотрена функция `multiplySequential(const Matrix& A, const Matrix& B, Matrix& C, int N)`. Она реализует классический трёхцикловый алгоритм умножения матриц (`C[i][j] += A[i][k] * B[k][j]`). При запуске программы в однопроцессном режиме (`world_size = 1`) вычисления выполняются именно этим методом.

Также была создана функция `printMatrix` для вывода матриц.

### **2. Реализация параллельного алгоритма умножения матриц**

Параллельная реализация начинается с инициализации MPI в функции `main`: вызовы `MPI_Init`, `MPI_Comm_size` и `MPI_Comm_rank` определяют

количество процессов и их глобальную нумерацию. Основной функционал вынесен в функцию `processTask`, которая отвечает за чтение размера матрицы, генерацию исходных данных, запуск последовательной или параллельной версии умножения.

Основная идея распараллеливания заключается в разбиении матриц на равные блоки (ленты), которые распределяются между процессами. Для упрощения реализации и сокращения объема коммуникаций алгоритм предполагает, что размер матрицы  $N$  равномерно делится на количество блоков по строкам и столбцам (`rowsPerProc` и `colsPerProc`). Это позволяет:

- Не реализовывать дополнения или паддинги, чтобы обработать «хвостовые» строки/столбцы;
- Минимизировать условные проверки при локальном умножении блоков;
- Сделать вычисления на каждом процессе максимально регулярными и симметричными.

Было решено использовать двухмерное полосное разбиение данных, которое позволяет распределить исходные матрицы между процессами без полного дублирования матриц на каждом процессе.

Чтобы обеспечить корректное распределение данных и удобную структуру коммуникаций, выбран механизм декартовой топологии MPI. Он позволяет представить множество процессов как двумерную сетку `dims[0] × dims[1]`, что идеально соответствует двумерной структуре умножаемых матриц.

Создание топологии выполняется следующим образом

```
int dims[2] = {0, 0};
```

```
MPI_Dims_create(world_size, 2, dims);
```

```
MPI_Cart_create(comm, 2, dims, periods, reorder, &cart_comm);
```

MPI автоматически подбирает размеры - как правило, близкие к квадратным ( $\sqrt{P} \times \sqrt{P}$ ), что минимизирует коммуникации в обеих координатах.

Далее выполняется расщепление декартовой сетки на одномерные подкоммуникаторы:

```
MPI_Cart_sub(cart_comm, {0,1}, &rowComm);
```

```
MPI_Cart_sub(cart_comm, {1,0}, &colComm);
```

rowComm — подкоммуникатор строки, используется для рассылки полос строк A. colComm — подкоммуникатор столбца, используется для рассылки полос столбцов B.

Такой выбор архитектуры важен по двум причинам:

- Разбиение данных соответствует топологии: каждая строка сетки отвечает блоку строк матрицы A, а каждый столбец — блоку столбцов матрицы B.
- Коммуникации становятся локальными: каждый процесс обменивается данными только со своей строкой или своим столбцом. Это существенно эффективнее, чем широковещательные рассылки или рассылки каждому узлу по очереди.

Эта структура полностью повторяет теоретический алгоритм, представленный на рисунке 1.

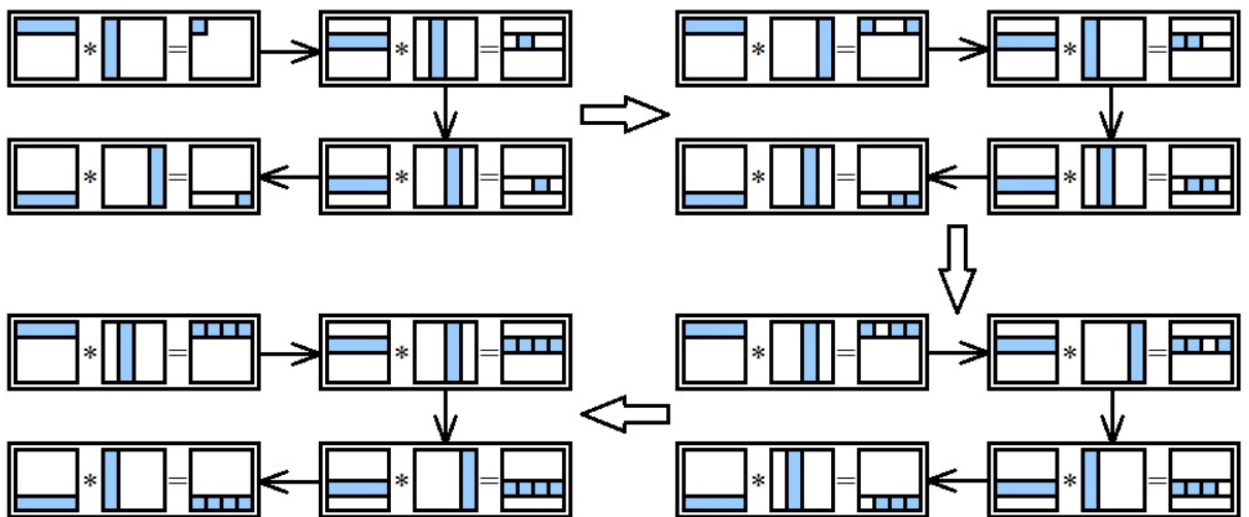


Рисунок 1. Визуализация ленточного алгоритма по строкам и столбцам.

Процесс обмена данными построен следующим образом:

1. Столбец 0 получает свои полосы строк матрицы A через Scatter - каждая строка получает ровно rowsPerProc строк A.
2. Строка 0 получает полосы столбцов матрицы B через Scatter - каждый столбец получает colsPerProc столбцов B.

3. Каждая строка транслирует свою часть  $A$  всем процессам строки через Broadcast.
4. Каждый столбец транслирует свою часть  $B$  всем процессам столбца через Broadcast.

Таким образом, каждая вычислительная нода получает два блока:

- $A_{local}$  — выдаётся из Scatter столбца 0 и распространяется по строке,
- $B_{local}$  — выдаётся из Scatter строки 0 и распространяется по столбцу.

Далее каждый процесс независимо выполняет умножение блока, не обращаясь к данным других процессов, с использованием функции `localMultiplyBlock`. Каждый процесс вычисляет только свой подблок  $C$  размером  $rowsPerProc \times colsPerProc$ .

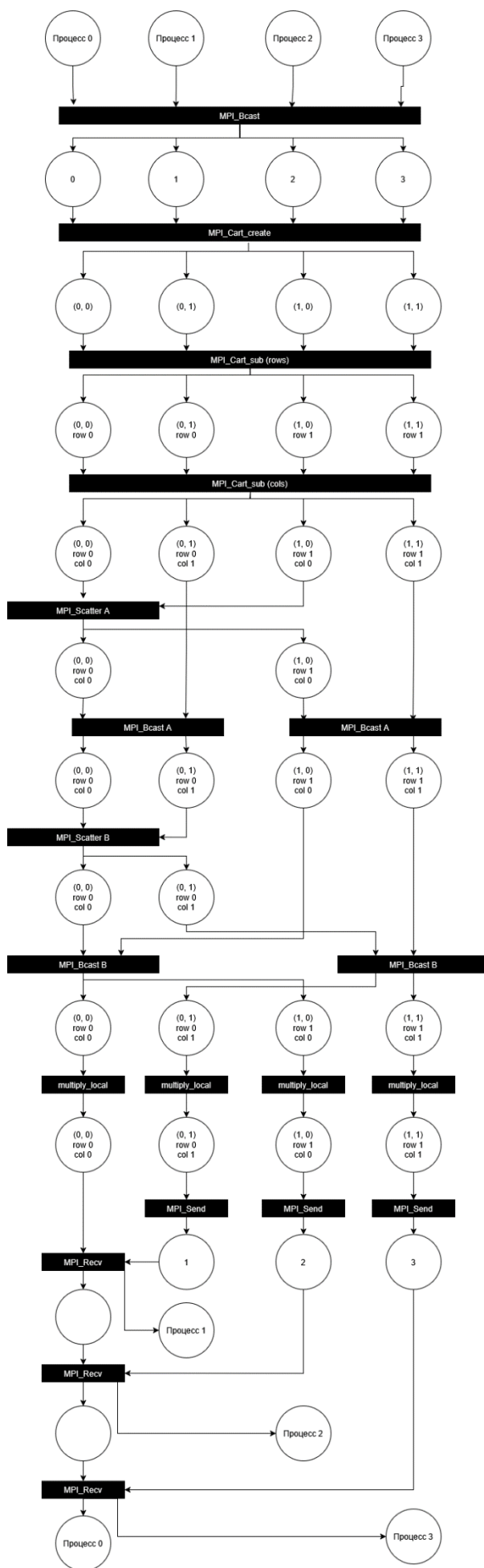
После завершения всех локальных умножений процесс с координатами (0,0) собирает результат через `MPI_Recv` из всех остальных процессов и восстанавливает глобальную матрицу  $C$  функцией `placeBlockIntoGlobal`.

Алгоритм масштабируется потому, что:

- Объём хранимых данных на процессе уменьшается пропорционально увеличению числа процессов.
- Коммуникации происходят внутри строк и столбцов, что ограничивает ширину передачи данных логарифмическим числом шагов.

После окончания вычислений каждый процесс освобождает созданные коммуникаторы, а на нулевом процессе происходит вывод результирующей матрицы.

Для визуального представления алгоритма нарисована сеть Петри, которая представлена на рисунке 2.



## Рисунок 2 – Сеть Петри.

Исследуем время выполнения программы при различном размере задачи для различного числа процессов. Результаты экспериментов представлены в таблице 1. Для каждого числа процессов и размеров поля значения приведены в усреднение на 50 запусков программы (время работы программы считалось по среднему времени завершения всех процессов). А также рассчитаем ускорение программы по формуле:

$$S_p(n) = \frac{T_1(n)}{T_p(n)}, \text{ где } p - \text{число процессов, а } n - \text{размеры поля}$$

Таблица 1. Результат работы программы.

| Размер<br>ность | Последовательный<br>алгоритм | 2 процесса |           | 5 процессов |           |
|-----------------|------------------------------|------------|-----------|-------------|-----------|
|                 |                              | время      | ускорение | время       | ускорение |
| 10              | 0.0084                       | 0.5175     | 0.0162    | 7.1467      | 0.0012    |
| 50              | 1.1247                       | 1.3571     | 0.8288    | 2.905       | 0.3872    |
| 100             | 9.4622                       | 4.3881     | 2.1563    | 10.6226     | 0.8908    |
| 200             | 69.8828                      | 19.421     | 3.5983    | 10.1278     | 6.9001    |
| 500             | 1020.24                      | 316.57     | 3.2228    | 121.197     | 8.418     |
| 1 000           | 8047.735                     | 2936.84    | 2.7403    | 2130.52     | 3.7774    |

| Размер<br>ность | 10 процессов |           | 25 процессов |           | 50 процессов |           |
|-----------------|--------------|-----------|--------------|-----------|--------------|-----------|
|                 | время        | ускорение | время        | ускорение | время        | ускорение |
| 10              | 10.0372      | 0.0008    | -            | -         | -            | -         |
| 50              | 8.0809       | 0.1392    | 43.7621      | 0.0257    | 145.921      | 0.0077    |
| 100             | 18.4496      | 0.5129    | 35.0732      | 0.2698    | 128.983      | 0.0734    |
| 200             | 20.0242      | 3.4899    | 45.2051      | 1.5459    | 79.9205      | 0.8744    |
| 500             | 168.037      | 6.0715    | 240.19       | 4.2476    | 340.298      | 2.9981    |
| 1 000           | 1379.81      | 5.8325    | 1473.485     | 5.4617    | 2019.89      | 3.9842    |

По данным из таблицы можно сделать несколько ключевых выводов о поведении программы с декартовой топологией. При малых размерах матриц наблюдается резкое увеличение времени при увеличении числа процессов (рисунок 3) и, как следствие, ускорение существенно меньше 1 (рисунок 5, 6).

Основные причины:

- Накладные расходы MPI. Каждое распределение данных (MPI\_Scatter) и последующая синхронизация (MPI\_Bcast) требуют передачи и копирования данных между процессами. Для маленьких матриц время этих операций может превосходить само вычисление.
- Создание виртуальных топологий (MPI\_Cart\_create, MPI\_Cart\_sub) добавляет фиксированную стоимость, которая критична для малых задач.
- На машине, на которой проводились эксперименты, 6 ядер: при использовании  $>6$  процессов создаются логические процессы, которые конкурируют за ядра CPU, что приводит к “переключению контекста” и увеличению времени.

В результате для  $N=10$  и  $N=50$  ускорения почти равны 0, что полностью согласуется с тем, что накладные расходы доминируют при малых задачах.

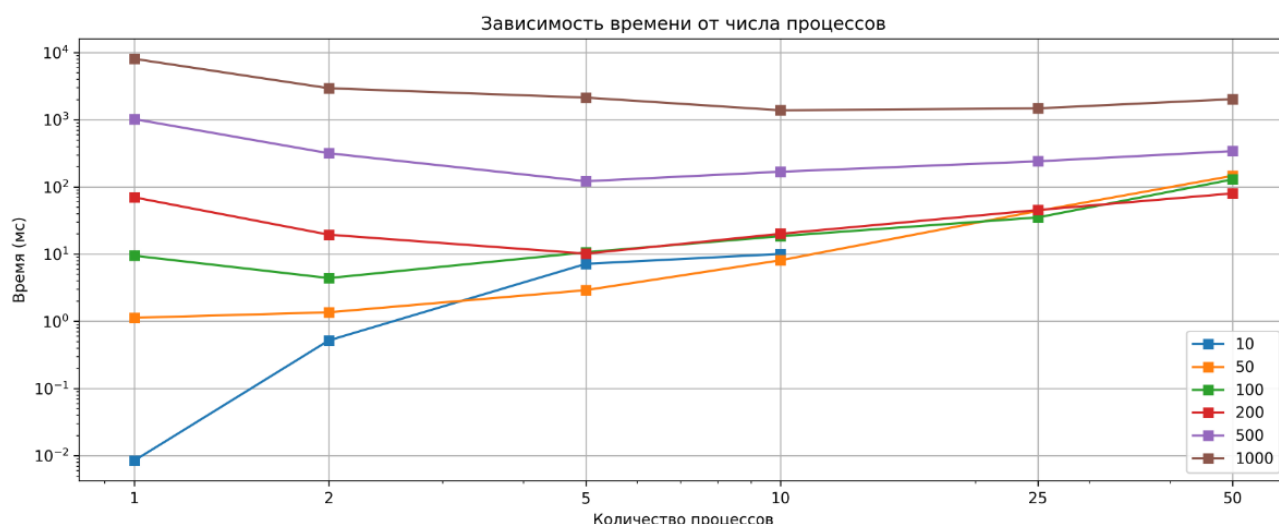


Рисунок 3. Зависимость времени от числа процессов.

На средних размерах матриц 100 - 200 наблюдается сложное поведение: ускорение сначала растёт, достигает пика, затем снова падает (рисунок 5, 6).



Для  $N=200$  на 5 процессах наблюдается максимальное ускорение  $S_p \approx 6.9$ , что связано с тем, что полезная работа (умножение блоков матрицы) начинает преобладать над накладными расходами. После достижения оптимума увеличение числа процессов приводит к повторному росту времени (рисунок 3). Одной из причин может быть уменьшение размера локальных блоков ( $\text{rowsPerProc} \times \text{colsPerProc}$ ), что при большом числе процессов приводит к увеличению числа MPI операций и уменьшению вычислительной плотности на каждом процессе. Также в снижение времени может играть роль оверлей (процессы  $>6$ ), когда процессы начинают конкурировать за одни и те же ядра. Для больших матриц схожее поведение со средними: ускорение растёт при увеличении числа процессов до некоторого оптимума, затем начинает падать (рисунок 5, 6). Ограничивающими факторами остаются коммуникационные накладные расходы MPI при рассылке полос матриц, транспонирование  $B$  и пересылка блоков между процессами, а также физическое ограничение количества ядер (6), что заставляет процессорные контексты переключаться при  $>6$  процессах.

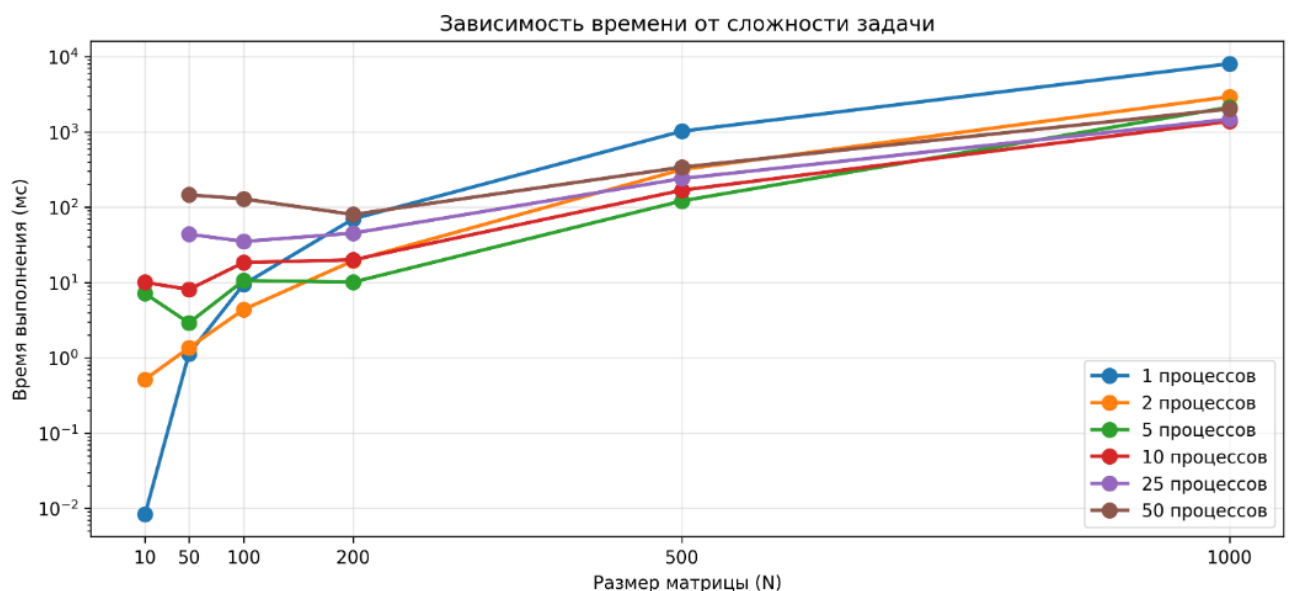


Рисунок 4. Зависимость от объема исходных данных.

Время выполнения зависит от объёма данных и числа процессов (рисунок 4). Для малых матриц накладные расходы на создание MPI-коммуникаторов, рассылку и сбор данных превышают затраты на вычисления, поэтому

увеличение числа процессов приводит к росту времени. Для средних и больших матриц разбиение на блоки позволяет лучше использовать параллельное выполнение, которое ускоряет вычисления.

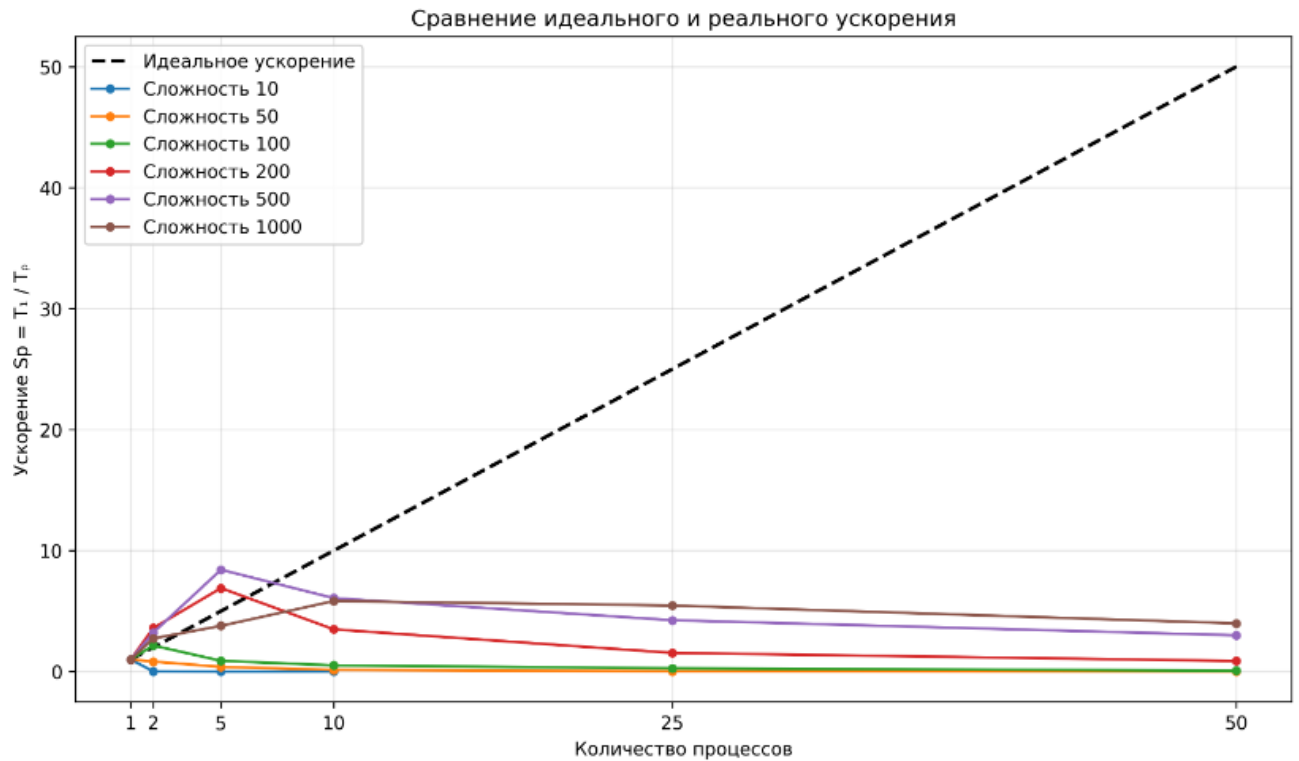


Рисунок 5. Ускорение программы для разного объема данных и числа процессов.

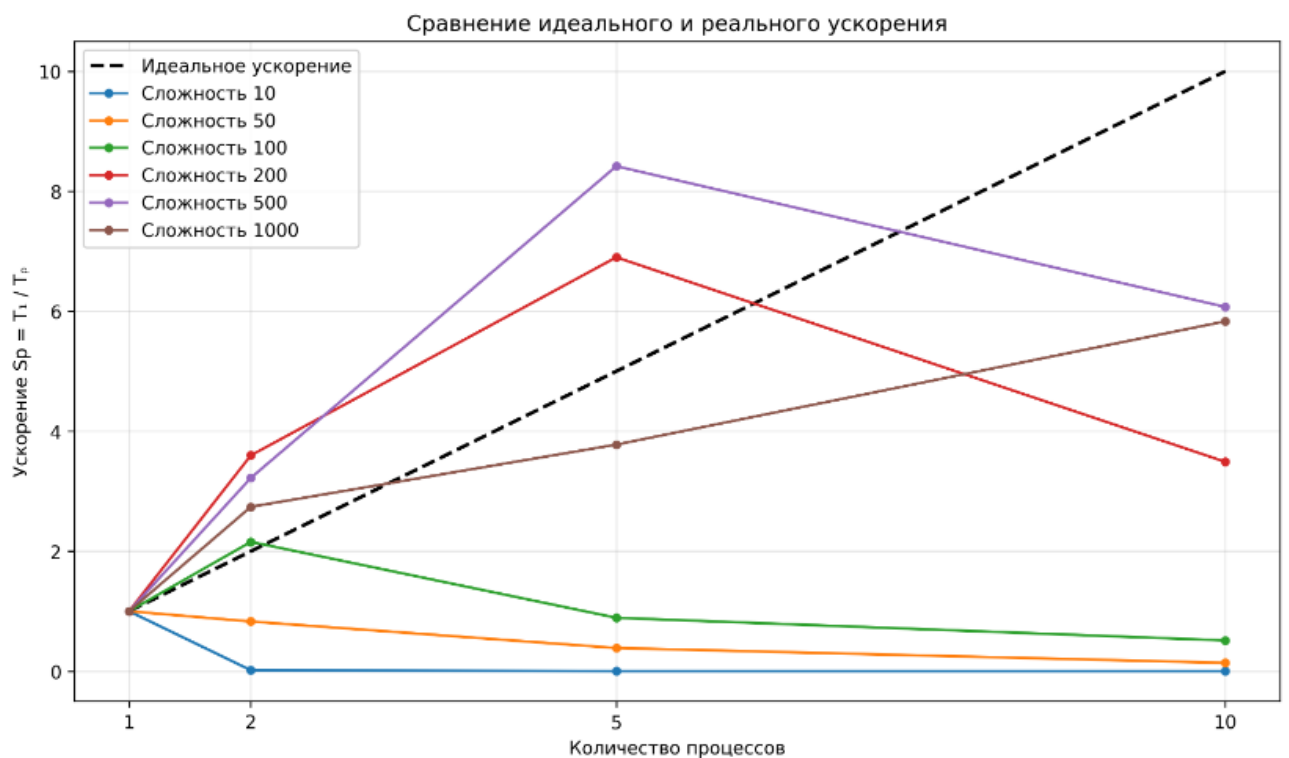


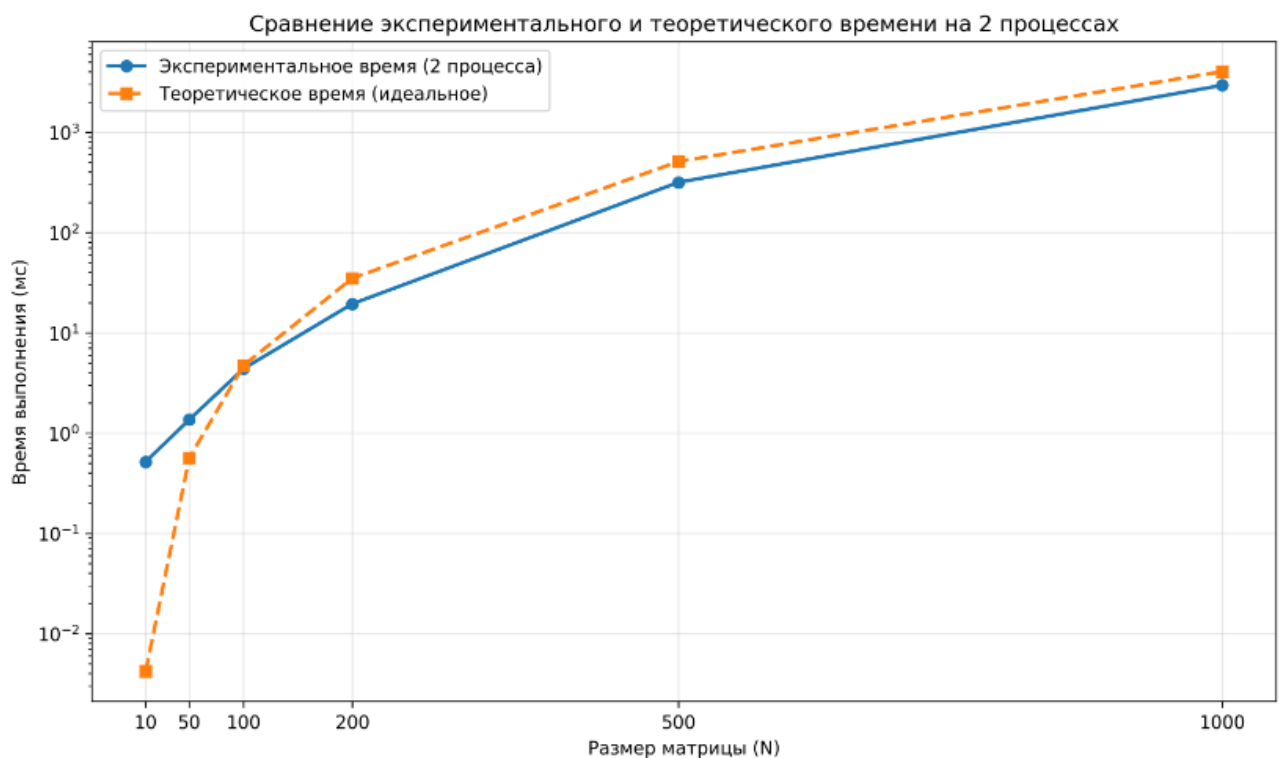
Рисунок 6. Ускорение программы для разного объема данных и числа процессов до 10

Идеальное ускорение, когда время сокращается прямо пропорционально числу процессов. На практике для малых  $N$  ускорение значительно меньше идеального ( $S_p \ll p$ ), для средних  $N$  ускорение превышает идеальное локально, для больших  $N$  ускорение стремится к идеальному, но ограничено физическими ядрами и накладными расходами MPI.

Время выполнения последовательного алгоритма напрямую зависит только от размера матрицы, т.к. количество операций  $m^3$ . Реализация параллельного алгоритма предполагает перемножения элементов линий матриц на каждом шаге. Так как каждый процесс используется  $m/p$  раз перемножая строку на столбец размера  $m$ , то сложность алгоритма без затрат на передачу данных будет равна  $m^3/p$ . Тогда ускорение представляется как:

$$S_p(m) = \frac{m^3}{\frac{m^3}{p}} = p$$

Сравним экспериментальные и теоретические оценки времени выполнения задачи на двух процессах. Результаты смотреть на рис. 5.



Исходный код программы находится в приложении А. Пример запуска программы в приложении В.

### **Выводы.**

В ходе выполнения лабораторной работы был успешно разработан, реализован и проанализирован параллельный алгоритм умножения квадратных матриц с использованием технологии MPI. Основным результатом работы стало создание эффективной параллельной реализации ленточного алгоритма с двумерным разбиением данных, демонстрирующей хорошую масштабируемость для задач среднего и большого размера.

Экспериментальные результаты демонстрируют четкую зависимость эффективности параллелизации от размера решаемой задачи. Для малых матриц (размером до 100 элементов) наблюдается значительное замедление работы при увеличении числа процессов, что объясняется преобладанием накладных расходов на организацию MPI-коммуникаций над полезными вычислениями. В частности, операции создания декартовой топологии, рассылки данных и синхронизации процессов оказываются более затратными, чем само умножение матриц.

Для средних матриц (размером 100-500 элементов) алгоритм демонстрирует оптимальную производительность с максимальным ускорением до 8.4 раза при использовании 5 процессов. Это свидетельствует о том, что для данного диапазона размеров задач достигается баланс между вычислительной нагрузкой и коммуникационными затратами. Реализованная схема двумерного разбиения данных с использованием декартовой топологии доказала свою эффективность для организации параллельных вычислений.

При работе с большими матрицами (свыше 500 элементов) наблюдается также хорошая масштабируемость алгоритма.

Проверка корректности работы алгоритма путем ручного вычисления отдельных элементов результирующих матриц подтвердила правильность реализации как последовательной, так и параллельной версий программы. Все

вычисления выполняются точно и соответствуют математическим правилам умножения матриц.

Таким образом, работа подтвердила, что эффективное распараллеливание вычислительных задач требует тщательного анализа как характеристик самой задачи, так и возможностей целевой вычислительной системы. Реализованный алгоритм может быть успешно применен для умножения матриц среднего и большого размера, демонстрируя при этом хорошие показатели ускорения и эффективности использования вычислительных ресурсов.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

```
main.cpp
#include <mpi.h>
#include <vector>
#include <iostream>
#include <random>
#include <algorithm>
#include <stdexcept>
#include <fstream>
#include <string>

#define MATRIX_SIZE 10000

using Matrix = std::vector<int>;

inline int& at(Matrix& m, int N, int i, int j) {
    return m[i * N + j];
}
inline int at(const Matrix& m, int N, int i, int j) {
    return m[i * N + j];
}

void printMatrix(const Matrix& M, int N, const std::string& name) {
    std::cout << "Матрица " << name << " (" << N << "x" << N << "):\n";
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            std::cout << at(M, N, i, j) << " ";
        }
        std::cout << "\n";
    }
}

Matrix generateRandomMatrix(int N, int minVal = 0, int maxVal = 9,
unsigned seed = 42) {
    Matrix M(N * N);
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> dist(minVal, maxVal);
    for (int i = 0; i < N * N; ++i) M[i] = dist(rng);
    return M;
}

void multiplySequential(const Matrix& A, const Matrix& B, Matrix& C, int
N) {
    C.assign(N * N, 0);
    for (int i = 0; i < N; ++i) {
        for (int k = 0; k < N; ++k) {
            int aik = at(A, N, i, k);
            for (int j = 0; j < N; ++j) {
                at(C, N, i, j) += aik * at(B, N, k, j);
            }
        }
    }
}
```

```

void checkDims(int dims[2], int N, int world_rank, MPI_Comm comm) {
    if (N % dims[0] != 0 || N % dims[1] != 0) {
        if (world_rank == 0) {
            std::cerr << "Ошибка: N = " << N
                << " должно делиться и на dims[0] = " << dims[0]
                << " и на dims[1] = " << dims[1] << "\n";
            std::cerr << "Выберите другое N или число процессов.\n";
        }
        MPI_Abort(comm, 1);
    }
}

void transposeMatrix(Matrix& M, int N) {
    for (int i = 0; i < N; ++i) {
        for (int j = i + 1; j < N; ++j) {
            std::swap(M[i * N + j], M[j * N + i]);
        }
    }
}

Matrix localMultiplyBlock(
    const Matrix& A_local,
    const Matrix& B_local,
    int N,
    int rowsPerProc,
    int colsPerProc
) {
    Matrix C_local(rowsPerProc * colsPerProc, 0);
    for (int i = 0; i < rowsPerProc; i++) {
        for (int k = 0; k < N; k++) {
            int aik = A_local[i * N + k];
            for (int j = 0; j < colsPerProc; j++) {
                C_local[i * colsPerProc + j] += aik * B_local[j * N + k];
            }
        }
    }
    return C_local;
}

void placeBlockIntoGlobal(
    const Matrix& local,
    Matrix& global,
    int N,
    int blockRows,
    int blockCols,
    int rowStart,
    int colStart
) {
    for (int i = 0; i < blockRows; ++i) {
        for (int j = 0; j < blockCols; ++j) {
            int globalRow = rowStart + i;
            int globalCol = colStart + j;
            at(global, N, globalRow, globalCol) = local[i * blockCols +
j];
        }
    }
}

```

```

void printLocalMatrix(
    const Matrix& C_local,
    int world_rank,
    int rowsPerProc,
    int colsPerProc,
    const std::string& matrixName = "C_local"
) {
    std::cout << "Process " << world_rank << " " << matrixName << " ("
        << rowsPerProc << "x" << colsPerProc << "):" << std::endl;

    for (int i = 0; i < rowsPerProc; ++i) {
        for (int j = 0; j < colsPerProc; ++j) {
            std::cout << C_local[i * colsPerProc + j] << " ";
        }
        std::cout << std::endl;
    }
    std::cout << std::endl;
}

void multiplyStriped2D(
    const Matrix& A,
    const Matrix& B,
    Matrix& C,
    int N,
    int world_size,
    int world_rank,
    MPI_Comm comm
) {
    // Создаём 2D-декартову топологию
    MPI_Comm cart_comm;
    int dimSize = 2;
    int dims[dimSize] = {0, 0};
    MPI_Dims_create(world_size, dimSize, dims);
    int periods[dimSize] = {0, 0};
    int reorder = 0;
    MPI_Cart_create(comm, dimSize, dims, periods, reorder, &cart_comm);

    // Проверяем делимость N на число блоков checkDims(dims, N,
world_rank);
    checkDims(dims, N, world_rank, comm);

    int cart_rank;
    MPI_Comm_rank(cart_comm, &cart_rank);

    int coords[dimSize];
    MPI_Cart_coords(cart_comm, cart_rank, dimSize, coords);
    int my_row = coords[0];
    int my_col = coords[1];
    bool is_grid_root = (my_row == 0 && my_col == 0);

    MPI_Comm rowComm;
    int remain_dims_row[2] = {0, 1};
    MPI_Cart_sub(cart_comm, remain_dims_row, &rowComm);

    MPI_Comm colComm;
    int remain_dims_col[2] = {1, 0};

```



```

MPI_Cart_sub(cart_comm, remain_dims_col, &colComm);

int row_rank, row_size;
MPI_Comm_rank(rowComm, &row_rank);
MPI_Comm_size(rowComm, &row_size);

int col_rank, col_size;
MPI_Comm_rank(colComm, &col_rank);
MPI_Comm_size(colComm, &col_size);

int col_root = 0, row_root = 0;

// Локальные буферы полос
int rowsPerProc = N / dims[0];
int colsPerProc = N / dims[1];
Matrix A_local(rowsPerProc * N);
Matrix B_local(colsPerProc * N);

// Рассылаем полосы A по строкам первого столбца
if (my_col == 0) {
    const int* sendbuf = is_grid_root ? A.data() : nullptr;
    MPI_Scatter(sendbuf, rowsPerProc * N, MPI_INT, A_local.data(),
rowsPerProc * N, MPI_INT, col_root, colComm);
    // printLocalMatrix(A_local, world_rank, rowsPerProc, N, "A
recieved " + std::to_string(my_row) + " " + std::to_string(my_col));
}
// Рассылаем полученные полосы A по всем столбцам в строке
MPI_Bcast(A_local.data(), rowsPerProc * N, MPI_INT, row_root,
rowComm);

// Рассылаем полосы B по столбцам первой строки
if (my_row == 0) {
    const int* sendbuf = is_grid_root ? B.data() : nullptr;
    MPI_Scatter(sendbuf, colsPerProc * N, MPI_INT, B_local.data(),
colsPerProc * N, MPI_INT, row_root, rowComm);
    // printLocalMatrix(B_local, world_rank, colsPerProc, N, "B
recieved " + std::to_string(my_row) + " " + std::to_string(my_col));
}
// Рассылаем полученные полосы B по всем строкам в столбце
MPI_Bcast(B_local.data(), colsPerProc * N, MPI_INT, col_root,
colComm);

// Локальное умножение: C_local = A_local * B_local
Matrix C_local = localMultiplyBlock(A_local, B_local, N, rowsPerProc,
colsPerProc);
// printLocalMatrixC(C_local, world_rank, rowsPerProc, colsPerProc,
// "C_local [" + std::to_string(my_row) + ", " +
std::to_string(my_col) + "]");

if (cart_rank != 0) {
    MPI_Send(C_local.data(), static_cast<int>(C_local.size()),
MPI_INT, 0, 2, cart_comm);
} else {
    C.assign(N * N, 0);
    Matrix recvC(rowsPerProc * colsPerProc);
    for (int pr_block = 0; pr_block < dims[0]; pr_block++) {
        for (int pc_block = 0; pc_block < dims[1]; pc_block++) {

```

```

        if (pr_block == 0 && pc_block == 0) {
            // Кладем свой локальный блок
            placeBlockIntoGlobal(C_local, C, N, rowsPerProc,
colsPerProc, 0, 0);
            continue;
        }

        int src_coords[dimSize] = {pr_block, pc_block};
        int src_rank;
        MPI_Cart_rank(cart_comm, src_coords, &src_rank);

        MPI_Recv(recvC.data(), static_cast<int>(recvC.size()),
MPI_INT, src_rank, 2, cart_comm, MPI_STATUS_IGNORE);

        int rowStart = pr_block * rowsPerProc;
        int colStart = pc_block * colsPerProc;
        placeBlockIntoGlobal(recvC, C, N, rowsPerProc,
colsPerProc, rowStart, colStart);
    }
}

MPI_Comm_free(&rowComm);
MPI_Comm_free(&colComm);
MPI_Comm_free(&cart_comm);
}

void getTimeStatistic(int rank, int size, double processTime) {
    std::vector<double> allTimes;
    if (rank == 0) {
        allTimes.resize(size);
    }
    MPI_Gather(&processTime, 1, MPI_DOUBLE,
allTimes.data(), 1, MPI_DOUBLE,
0, MPI_COMM_WORLD);

    if (rank == 0) {
        double avgTime = std::accumulate(allTimes.begin(),
allTimes.end(), 0.0) / size;

        std::cout << "\n== РЕЗУЛЬТАТЫ ДЛЯ " << size << " ПРОЦЕССОВ ==>
<< std::endl;
        std::cout << "Среднее время: " << avgTime << " мс" << std::endl;
        std::string filename = "results_" + std::to_string(MATRIX_SIZE) +
".txt";
        std::ofstream file(filename, std::ios::app);
        if (file.is_open()) {
            file << size << " " << avgTime << std::endl;
            file.close();
            std::cout << "Результаты записаны в " << filename <<
std::endl;
        } else {
            std::cout << "Ошибка открытия файла!" << std::endl;
        }
    }
}

```

```

void processTask(int world_size, int world_rank, MPI_Comm comm) {
    int N = MATRIX_SIZE;
    Matrix A, B, C;

    if (world_rank == 0) {
        // std::cout << "Введите размер матрицы N: ";
        // std::cin >> N;
        // if (N <= 0) {
        //     std::cerr << "N должно быть положительным\n";
        //     MPI_Abort(comm, 3);
        // }
        A = generateRandomMatrix(N, 0, 9, 42);
        B = generateRandomMatrix(N, 0, 9, 4242);
        // printMatrix(A, N, "A");
        // printMatrix(B, N, "B");
    }
    MPI_Bcast(&N, 1, MPI_INT, 0, comm);

    double startTime = MPI_Wtime();
    if (world_size == 1) {
        multiplySequential(A, B, C, N);
    } else {
        if (world_rank == 0) {
            transposeMatrix(B, N);
        }
        multiplyStriped2D(A, B, C, N, world_size, world_rank, comm);
    }
    double endTime = MPI_Wtime();
    double processTime = (endTime - startTime) * 1000;

    getTimeStatistic(world_rank, world_size, processTime);

    if (world_rank == 0) {
        // printMatrix(B, N, "Bt");
        // printMatrix(C, N, "C (результат)");
    }
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int world_size = 0, world_rank = 0;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    processTask(world_size, world_rank, MPI_COMM_WORLD);

    MPI_Finalize();
    return 0;
}

```

## ПРИЛОЖЕНИЕ Б

### РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

Матрица А (10x10):

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 3 | 7 | 9 | 1 | 7 | 7 | 5 | 5 | 1 | 4 |
| 1 | 0 | 0 | 4 | 8 | 3 | 6 | 1 | 7 | 6 |
| 0 | 0 | 9 | 7 | 8 | 9 | 2 | 0 | 1 | 9 |
| 1 | 6 | 3 | 6 | 5 | 0 | 4 | 0 | 2 | 5 |
| 6 | 3 | 1 | 0 | 2 | 9 | 3 | 2 | 4 | 0 |
| 7 | 6 | 1 | 3 | 5 | 9 | 5 | 4 | 0 | 8 |
| 6 | 6 | 1 | 4 | 0 | 0 | 9 | 9 | 9 | 5 |
| 8 | 3 | 3 | 0 | 0 | 2 | 6 | 2 | 4 | 6 |
| 1 | 6 | 4 | 8 | 0 | 1 | 9 | 3 | 2 | 1 |
| 6 | 7 | 3 | 4 | 5 | 2 | 5 | 5 | 1 | 0 |

Матрица В (10x10):

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 3 | 8 | 9 | 2 | 9 | 4 | 2 | 3 | 7 | 1 |
| 6 | 8 | 5 | 5 | 3 | 6 | 7 | 1 | 5 | 9 |
| 5 | 0 | 1 | 4 | 4 | 6 | 1 | 4 | 0 | 2 |
| 9 | 3 | 6 | 1 | 6 | 1 | 7 | 9 | 6 | 1 |
| 3 | 3 | 2 | 6 | 4 | 9 | 1 | 0 | 8 | 2 |
| 6 | 3 | 0 | 5 | 8 | 7 | 2 | 6 | 9 | 4 |
| 5 | 9 | 2 | 9 | 9 | 2 | 7 | 3 | 4 | 8 |
| 3 | 5 | 3 | 9 | 6 | 7 | 8 | 3 | 6 | 6 |
| 8 | 5 | 4 | 4 | 8 | 3 | 0 | 8 | 8 | 6 |
| 7 | 3 | 9 | 1 | 5 | 7 | 9 | 2 | 8 | 2 |

Матрица С (результат) (10x10):

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 244 | 212 | 156 | 253 | 277 | 297 | 203 | 149 | 271 | 211 |
| 212 | 165 | 146 | 166 | 235 | 183 | 148 | 146 | 256 | 141 |
| 267 | 122 | 156 | 167 | 253 | 266 | 179 | 185 | 275 | 117 |
| 194 | 150 | 149 | 129 | 172 | 158 | 167 | 113 | 185 | 131 |
| 154 | 162 | 102 | 149 | 218 | 161 | 91  | 126 | 210 | 135 |
| 251 | 244 | 216 | 215 | 304 | 275 | 240 | 155 | 326 | 192 |
| 274 | 294 | 235 | 253 | 332 | 213 | 263 | 200 | 298 | 256 |
| 179 | 196 | 178 | 147 | 237 | 162 | 156 | 119 | 205 | 145 |
| 214 | 192 | 135 | 178 | 219 | 131 | 202 | 157 | 172 | 179 |
| 186 | 212 | 155 | 197 | 230 | 195 | 176 | 123 | 217 | 173 |